Cloud Computing Assignment: Final Technical Report

Student Name: Abhinav Kumar Jha
Roll Number: 230101044
Date: 8th November 2026

Task 1: Microservice Architecture & Containerization

1.1 Architecture

The system is built as a cloud-native microservice utilizing FastAPI for asynchronous HTTP request handling and Uvicorn as the ASGI server. To decouple the application logic from heavy machine learning model weights, the architecture relies on an external S3-compatible object store (SeaweedFS). At runtime, the container fetches the required model artifacts (`model-v1.tar.gz`) before binding to port 8000.

1.2 Implementation

The application was containerized using a `Dockerfile` that packages Python 3.11, PyTorch, and the FastAPI application layer. The build process ensures that the container image remains lightweight by excluding model `.safetensors` files from the Docker build context. Instead, `upload_s3.py` seeds the local SeaweedFS instance, and the container pulls these dynamically on startup.

1.3 Results

- **Measured Result:** The Docker image built successfully and was imported into the local `k3d` cluster using `k3d image import`. Container initialization logs (`cold_start_model_download.log`) confirm the model downloads and extracts in ~0.05 seconds locally, followed by a successful `200 OK` on the `/healthz` endpoint.
- **Assumption:** It was assumed that the network bandwidth between the k3d worker nodes and the SeaweedFS pods would be near-instantaneous (loopback speeds), which held true in the local environment.

1.4 Limitations & Improvements

- **Limitation:** Tying container startup to a synchronous HTTP download from an S3 bucket introduces a single point of failure and increases cold-start latency.
- **Improvement:** Implement an `InitContainer` to handle the model download to a shared `emptyDir` volume, allowing the main application container to start only after weights are verified on disk.

Task 2: Kubernetes Orchestration & Resource Quotas

2.1 Architecture

The deployment targets a local `k3d` Kubernetes cluster with distinct namespace isolation. The application resides in the `pipeline` namespace, while stateful services (SeaweedFS) reside in the `data` namespace.

The architecture heavily relies on Linux control groups (`cgroups`) to enforce resource boundaries via Kubernetes `requests` and `limits`.

2.2 Implementation

The `Deployment` manifest (`k8s/deployment.yaml`) was configured with a strict ceiling to simulate constrained edge environments:

- **Requests:** `250m` CPU, `512Mi` Memory
- **Limits:** `1000m` (1 Core) CPU, `1Gi` (1024Mi) Memory A Horizontal Pod Autoscaler (HPA) was also applied to scale the service from 1 to 3 replicas if CPU utilization exceeds 70%.

2.3 Results

- **Measured Result:** Telemetry captured via `kubectl top pods` showed the pod consuming **1502m CPU** (triggering CFS throttling) and **858 MiB Memory** under load.
- **Assumption:** We assumed the model inference engine (Transformers/PyTorch) would peak at roughly 600-700 MiB of RAM.
- **Conclusion:** The measured memory (858 MiB) ran closer to the 1 GiB limit than assumed, but successfully avoided triggering the Linux kernel's Out-Of-Memory (OOM) Killer.

2.4 Limitations & Improvements

- **Limitation:** CPU throttling (exceeding 1000m) significantly degraded inference response times during concurrent requests.
- **Improvement:** Apply model quantization (e.g., INT8 or ONNX format) to reduce the memory footprint by 50% and decrease CPU cycle requirements per inference request.

Task 3: CI/CD Pipeline & MLOps Quality Gates

3.1 Architecture

The Continuous Integration pipeline is orchestrated via GitHub Actions (`.github/workflows/ci.yml`). It shifts away from standard software pipelines by introducing an MLOps "Quality Gate" that validates the probabilistic outputs of the LLM against a deterministic schema.

3.2 Implementation

The pipeline executes a 5-stage sequential workflow:

1. **Static Linting:** `ruff` checks Python PEP-8 compliance.
2. **Unit Testing:** `pytest` verifies API routing without loading the model (`SKIP_MODEL_LOAD=1`).
3. **Model Evaluation:** The runner downloads `model-v1.tar.gz` from GitHub Releases and runs inference over `data/validation.jsonl`.
4. **Quality Gate:** `tools/score.py` asserts that schema validity must be $\ge$ 95%.
5. **Security Scanning:** `gitleaks` audits the commit history for hardcoded AWS/S3 secrets.

3.3 Results

- **Measured Result:** The fine-tuned model (`student_v1`) scored **100.0% JSON validity**, successfully passing the 95% gate. Intent accuracy was measured at 38.0%, and place accuracy at 87.0%. When simulating a failure with an untrained model (`Pythia-70m`), the pipeline correctly blocked the deployment.
- **Assumption:** It was assumed that Gitleaks would pass cleanly, which was verified as no AWS keys were hardcoded into the pushed codebase.

3.4 Limitations & Improvements

- **Limitation:** The CI runner downloads multi-gigabyte models on every push, making the pipeline slow and expensive in a real-world cloud runner environment.
- **Improvement:** Implement GitHub Actions caching for `artifacts/models/` to reuse weights across pipeline runs unless the release tag changes.

Task 4: Load Testing & Zero-Downtime Resilience

4.1 Architecture

System resilience is achieved through a combination of Kubernetes `RollingUpdate` deployment strategies and robust readiness/liveness probes (`GET /healthz`). Load simulation is handled externally via Locust.

4.2 Implementation

- **Locust:** Configured to spawn 10 concurrent users with a spawn rate of 2 users/sec for a 30-second duration.
- **Rolling Updates:** Configured `maxSurge: 25%` and `maxUnavailable: 0`. The update was triggered by injecting `MODEL_VERSION=v2` into the environment.

4.3 Results

- **Measured Result (Load Test):** Processed **126/126 requests** successfully with 0 failures under the 1 CPU / 1 GiB limit.
- **Measured Result (Rollout/Rollback):** The transition to `v2` completed without dropping connections. Upon executing `kubectl rollout undo`, HTTP response headers (`X-Model-Version`) immediately confirmed traffic routing reverted to `v1` seamlessly.
- **Assumption:** Assumed that the `/healthz` probe accurately reflects model readiness. Since `healthz` checks the FastAPI status *after* model extraction, this assumption held true, preventing premature traffic routing.

4.4 Limitations & Improvements

- **Limitation:** The single `/healthz` endpoint serves as both a liveness and readiness probe.
- **Improvement:** Decouple probes. Liveness should check if the API is running (lightweight), while Readiness should verify if the model weights are loaded and ready for inference (heavy).

Task 5: VPC Subnetting & Network Security (NACLs)

5.1 Architecture

The network topology is designed around a `10.20.0.0/16` Virtual Private Cloud (VPC), subdivided using Variable Length Subnet Masking (VLSM). Security is enforced via stateless Network Access Control Lists (NACLs).

5.2 Implementation

- **App Subnet:** `10.20.0.0/22` (1,022 hosts)
- **Public Subnet:** `10.20.4.0/24` (254 hosts)
- **Data Subnet:** `10.20.5.0/25` (126 hosts)
- **Management Subnet:** `10.20.5.128/27` (30 hosts) The `network/nacl.json` file implements strict inbound/outbound rules evaluated in ascending priority.

5.3 Results

- **Measured Result:** The `tools/nacl_sim.py` simulator validated the rule logic. Packet 1 from the malicious IP `198.51.100.50` was successfully intercepted and **DENIED** by Rule 50, while standard HTTP traffic was **ALLOWED** by Rule 100.
- **Assumption:** Because NACLs are stateless, it was assumed that return traffic must be explicitly allowed. We successfully implemented this by explicitly opening outbound ephemeral ports (`1024-65535`).

5.4 Limitations & Improvements

- **Limitation:** Managing stateless NACL rules for ephemeral return ports is complex and prone to human error, potentially opening the network to vulnerabilities.
- **Improvement:** Migrate to stateful Security Groups (or Kubernetes `NetworkPolicy` via Calico/Cilium), which automatically track connection states and allow return traffic implicitly.